

# Addressing & Operand Encoding

## Instruction Format Specification for a Two-Pass Assembler Targeting 32-bit x86 (IA-32)

This section explains how an assembler represents registers and memory operands in 32-bit x86 machine code. The main components are register encoding, the ModR/M byte, the SIB byte, and displacement.

Main question: How does the assembler represent the operands and memory address?

### 1. 32-bit Register Encoding

The general-purpose 32-bit registers are identified using 3-bit codes. These codes are reused in the ModR/M and SIB fields.

| Binary | Register |
|--------|----------|
| 000    | EAX      |
| 001    | ECX      |
| 010    | EDX      |
| 011    | EBX      |
| 100    | ESP      |
| 101    | EBP      |
| 110    | ESI      |
| 111    | EDI      |

For example: EAX = 000, EBX = 011, and ESI = 110.

### 2. ModR/M Byte

The ModR/M byte tells the processor whether an operand is a register or a memory operand, and which registers are involved.

Bit layout:

7 6 | 5 4 3 | 2 1 0  
MOD | REG | R/M

| Field | Size   | Purpose                                           |
|-------|--------|---------------------------------------------------|
| MOD   | 2 bits | Selects the addressing mode                       |
| REG   | 3 bits | Selects a register or acts as an opcode extension |
| R/M   | 3 bits | Selects a register or memory-addressing form      |

### 3. MOD Field

| MOD | Meaning                                          |
|-----|--------------------------------------------------|
| 00  | Memory addressing, normally without displacement |
| 01  | Memory addressing with signed 8-bit displacement |
| 10  | Memory addressing with 32-bit displacement       |
| 11  | Register addressing                              |

So, MOD = 11 means the operand is a register. MOD = 01 or 10 means memory is being addressed with a displacement.

## 4. REG and R/M Fields

REG uses the same 3-bit register codes listed earlier. R/M has two roles. When MOD = 11, R/M directly selects a register. When MOD is 00, 01, or 10, R/M selects the memory-addressing form.

## 5. ModR/M Lookup Table

| MOD | R/M     | Addressing form     |
|-----|---------|---------------------|
| 00  | 000     | [EAX]               |
| 00  | 001     | [ECX]               |
| 00  | 010     | [EDX]               |
| 00  | 011     | [EBX]               |
| 00  | 100     | SIB follows         |
| 00  | 101     | [disp32]            |
| 00  | 110     | [ESI]               |
| 00  | 111     | [EDI]               |
| 01  | 000     | [EAX + disp8]       |
| 01  | 001     | [ECX + disp8]       |
| 01  | 010     | [EDX + disp8]       |
| 01  | 011     | [EBX + disp8]       |
| 01  | 100     | [SIB + disp8]       |
| 01  | 101     | [EBP + disp8]       |
| 01  | 110     | [ESI + disp8]       |
| 01  | 111     | [EDI + disp8]       |
| 10  | 000–111 | [Register + disp32] |
| 11  | 000     | EAX                 |
| 11  | 001     | ECX                 |
| 11  | 010     | EDX                 |
| 11  | 011     | EBX                 |
| 11  | 100     | ESP                 |
| 11  | 101     | EBP                 |
| 11  | 110     | ESI                 |
| 11  | 111     | EDI                 |

Important cases: MOD = 00, R/M = 100 means a SIB byte follows. MOD = 00, R/M = 101 means a 32-bit displacement is used without a base register.

## 6. Register Addressing

Example: MOV EBX, EAX

MOD = 11, REG = EBX = 011, R/M = EAX = 000

ModR/M = 11 011 000 = 11011000 = D8

The exact opcode is selected separately. Here, the important point is that MOD = 11 makes both operands register operands.

## 7. Memory Addressing

A memory operand can use a register as its base address. For example, [EBX] means that the memory location whose address is stored in EBX is accessed.

For MOV EAX, [EBX]:

MOD = 00, REG = EAX = 000, R/M = EBX = 011

ModR/M = 00 000 011 = 00000011 = 03

## 8. Displacement Addressing

A displacement is an offset added to a base address. IA-32 commonly uses an 8-bit or 32-bit displacement.

| Type   | Use                 | Example                  |
|--------|---------------------|--------------------------|
| disp8  | Signed -128 to +127 | [EBX + 0x10] → 10        |
| disp32 | 32-bit displacement | 0x12345678 → 78 56 34 12 |

The 32-bit example is stored in Little-Endian order: the least significant byte comes first.

## 9. SIB Byte

The SIB (Scale-Index-Base) byte is used for more complex memory addresses, especially when an index register is multiplied by 1, 2, 4, or 8.

Bit layout:

7 6 | 5 4 3 | 2 1 0  
SCALE | INDEX | BASE

| Field | Size       | Purpose                      |
|-------|------------|------------------------------|
| SCALE | 2 bits     | Selects the index multiplier |
| INDEX | 3 bits     | Selects the index register   |
| BASE  | 3 bits     | Selects the base register    |
| SCALE | Multiplier |                              |
| 00    | ×1         |                              |
| 01    | ×2         |                              |
| 10    | ×4         |                              |
| 11    | ×8         |                              |

INDEX and BASE use the same 3-bit register codes. A common address form is: Base + Index × Scale + Displacement.

## 10. Complex Addressing Example

Consider: MOV EAX, [EBX + ESI\*4 + 0x10]

| Part         | Value | Encoding |
|--------------|-------|----------|
| Destination  | EAX   | 000      |
| Base         | EBX   | 011      |
| Index        | ESI   | 110      |
| Scale        | 4     | 10       |
| Displacement | 0x10  | 10       |

### 10.1 Constructing ModR/M

A SIB byte is needed, so R/M = 100. The displacement 0x10 fits in signed 8 bits, so MOD = 01. The destination register EAX gives REG = 000.

MOD REG R/M  
01 000 100

ModR/M = 01000100 = 44

## 10.2 Constructing SIB

SCALE = 10 → ×4  
INDEX = 110 → ESI  
BASE = 011 → EBX

SIB = 10 110 011 = 10110011 = B3

## 10.3 Adding the Displacement

The displacement is 0x10 and it fits in 8 bits, so disp8 = 10.

## 10.4 Final Encoding

| Component                | Value |
|--------------------------|-------|
| Opcode for MOV r32,r/m32 | 8B    |
| ModR/M                   | 44    |
| SIB                      | B3    |
| disp8                    | 10    |

**Final machine code: 8B 44 B3 10**

This shows how the memory operand [EBX + ESI\*4 + 0x10] is converted into the ModR/M byte, SIB byte, and displacement.

## 11. Summary

The assembler converts an assembly operand into small binary fields. Register operands use 3-bit register codes. The ModR/M byte selects the main register or memory-addressing form. If the address contains a scaled index, a SIB byte is added. A displacement is included when the address needs an offset.

Flowchart :

Assembly operand → identify registers/addressing mode → build ModR/M → add SIB if needed → add displacement → combine with the instruction opcode.